VISIONROOT — AI ENGINEER TRAINEE

Daily Submission Report — Day 11

Date: 02 October 2026
Week: Week 03 — Production AI Engineering
Topic: AI API Development using FastAPI

1. Objective

The objective of Day 11 was to develop a REST API for AI-powered functionalities using FastAPI. The main focus was to understand API development, request validation, authentication basics, error handling, logging, API documentation, and secure configuration management.

2. What I Learned

During Day 11, I learned the following concepts:

- **REST API principles:** Learned how applications communicate through HTTP methods and endpoints, and how APIs return structured responses.
- **FastAPI:** Learned how to create an API application and define endpoints for AI-powered functionalities.
- **Request and response models:** Learned how to use Pydantic models to define expected request data and validate incoming requests.
- **Input validation:** Learned how to validate user input using field constraints such as minimum and maximum string lengths.
- **Authentication basics:** Learned how an API key can be used to restrict access to API endpoints.
- **Error handling:** Learned how to handle errors that occur when communicating with an external AI service.
- **Logging:** Learned how to record application events and errors for debugging and monitoring.
- **API documentation:** Learned how FastAPI automatically generates interactive API documentation using Swagger UI and OpenAPI.
- **Environment variables:** Learned how to store sensitive configuration values outside the Python source code using a .env file.

3. What I Implemented

I developed a FastAPI-based AI API application in the following directory:

Week-03/Day-11/practical-task/

The implementation included the following components:

a. FastAPI Application

Created a FastAPI application named VisionRoot AI API with a title, description, and version number.

b. AI Chat Endpoint

Implemented the following endpoint:

POST /api/ai/chat

This endpoint accepts a user message, sends it to the Gemini API, and returns the AI-generated response.

c. AI Summarization Endpoint

Implemented the following endpoint:

POST /api/ai/summarize

This endpoint accepts text and sends a summarization prompt to the AI model.

d. AI Question-Answering Endpoint

Implemented the following endpoint:

POST /api/ai/ask

This endpoint accepts a question and sends it to the AI model to generate an answer.

e. Request Validation

Created Pydantic request models for chat, summarization, and question-answering operations. Added input constraints to prevent empty or excessively long text from being processed.

f. API-Key Authentication

Implemented a basic API-key verification mechanism using the x-api-key request header. Requests with a missing or incorrect key are rejected with an HTTP 401 Unauthorized response.

g. Error Handling and Logging

Added exception handling around Gemini API requests and used Python's logging module to record incoming requests and failures.

h. Secure Configuration

Configured the application to load the Gemini API key and the application API key from environment variables using python-dotenv. Added .env and virtual-environment entries to .gitignore to reduce the risk of accidentally committing sensitive information.

i. API Documentation

Used FastAPI's automatically generated Swagger UI and OpenAPI documentation to inspect and test the available endpoints.

4. What Worked

The following components were successfully verified:

- The FastAPI application started successfully using Uvicorn.
- The application was accessible at http://127.0.0.1:8000.
- The Swagger documentation page at http://127.0.0.1:8000/docs loaded successfully.
- The OpenAPI schema at http://127.0.0.1:8000/openapi.json loaded successfully.
- The chat endpoint received requests, as confirmed by the application logs.
- The application logged incoming chat requests and errors.
- The Gemini client was initialized, and requests reached the Gemini API.
- The exception-handling mechanism caught the error returned by the external AI service.

5. What Failed

During testing of the chat endpoint, the Gemini API returned the following error:

503 Service Unavailable

The error message indicated that the selected Gemini model was experiencing high demand.

As a result, the AI-generated response could not be returned successfully, and the application returned an HTTP 500 Internal Server Error.

This showed that an application can be configured correctly while an external AI service is temporarily unavailable.

6. How I Solved / Investigated the Problem

I investigated the issue using the terminal output and application logs.

The logs confirmed that:

1. The FastAPI server was running successfully.
2. The chat request reached the application.
3. The Gemini API request was sent successfully at the HTTP communication level.
4. The Gemini service returned a 503 Service Unavailable error.
5. The application's exception handler recorded the failure in the logs.

Based on this investigation, I identified the problem as a temporary error from the external AI service rather than a failure to start the FastAPI application.

The next troubleshooting step is to check the Gemini API's available models and select a currently supported model if necessary. The external service should then be tested again.

7. What I Would Improve

To make the application more suitable for production, I would improve the following areas:

- **Consistent error responses:** Return a predictable JSON structure for both successful requests and errors.
- **Appropriate HTTP status codes:** Handle upstream errors such as 503 Service Unavailable and 429 Too Many Requests more accurately instead of returning 500 for every exception.
- **Retry handling:** Introduce limited retries with appropriate delays for temporary AI-service failures.
- **Improved logging:** Record useful error information without exposing API keys, personal information, or other sensitive data.
- **Authentication security:** Use a securely generated application API key and protect it using environment variables.

- **Configuration validation:** Check required environment variables at application startup.
- **Testing:** Add tests for successful requests, invalid inputs, missing API keys, and AI-service failures.
- **Documentation:** Improve the README file with installation instructions, environment-variable configuration, endpoint descriptions, and example requests and responses.
- **API reliability:** Test the application with both successful AI responses and simulated external-service failures.

8. Tools and Technologies Used

- Python
- FastAPI
- Uvicorn
- Pydantic
- Google Gemini API
- Google Gen AI Python SDK
- python-dotenv
- Python logging
- Swagger UI
- OpenAPI
- Visual Studio Code
- PowerShell
- Git and GitHub

10. Testing Evidence

The following evidence can be included with the daily submission:

1. Screenshot of the project structure.

2. Screenshot showing Uvicorn starting successfully.
3. Screenshot of the Swagger UI documentation page.
4. Screenshot of request-validation testing.
5. Screenshot of API-key authentication testing.
6. Screenshot of the chat endpoint logs showing the Gemini 503 error.

Note: Include only screenshots of tests that were actually performed. A successful AI-response screenshot should be added after a successful request has been verified.

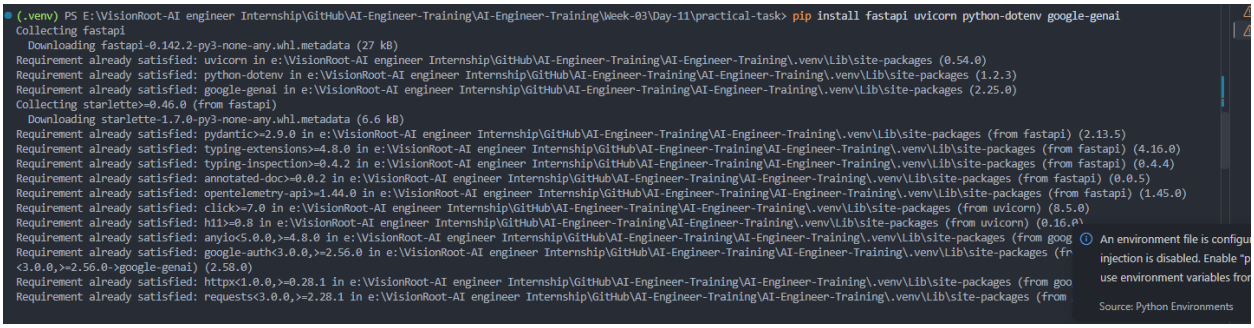
12. Conclusion

Day 11 provided practical experience in developing an AI-powered REST API using FastAPI. I implemented endpoints for chat, summarization, and question-answering, together with request validation, basic API-key authentication, logging, error handling, and automatic API documentation.

Although the Gemini chat request encountered a temporary 503 Service Unavailable error, investigating the issue helped me understand the importance of handling external-service failures and designing reliable AI applications.

This task strengthened my understanding of the steps required to move an AI prototype toward a more structured, secure, maintainable, and production-oriented application.

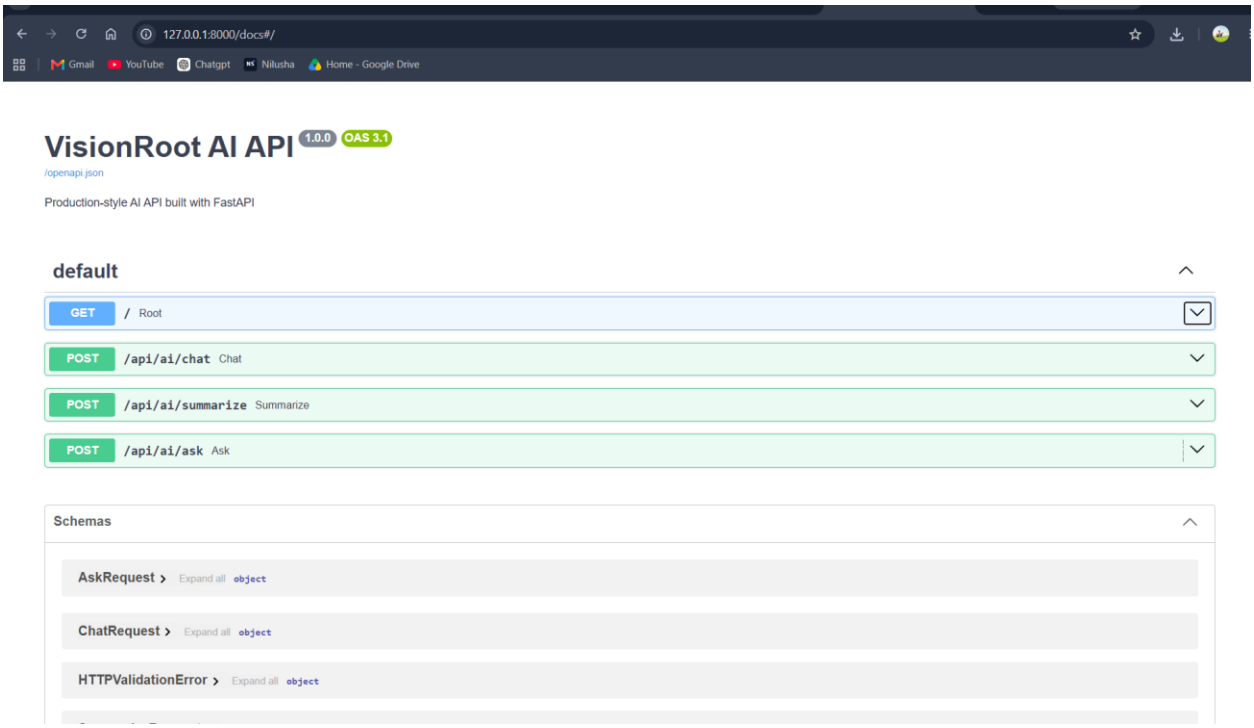
Status: Core API setup and documentation verified. Gemini response testing requires further verification because the selected model returned a 503 Service Unavailable error.



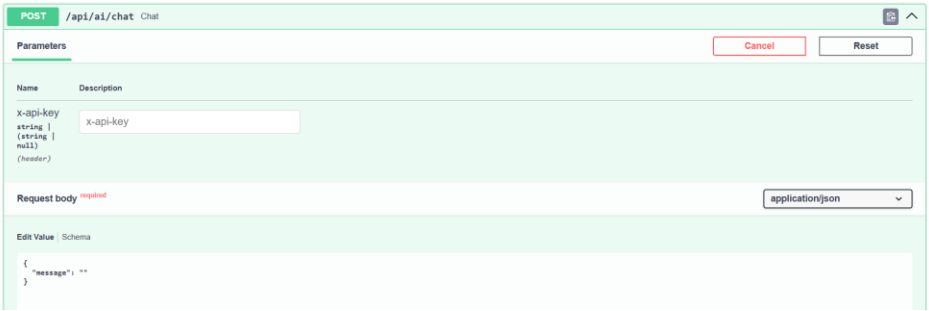
Start the FastAPI server

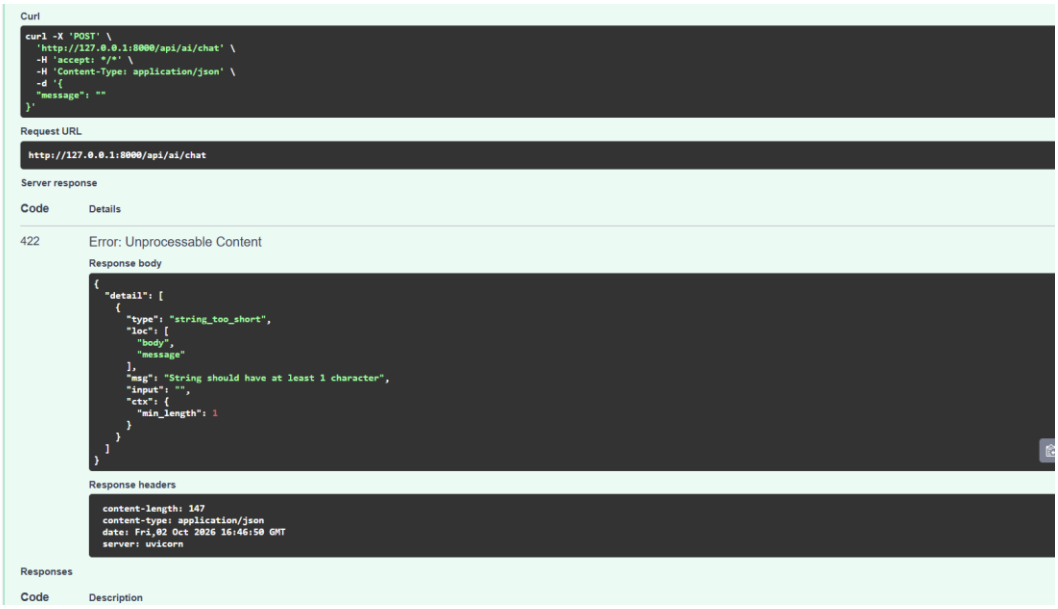
```
[notice] A new release of pip is available: 26.1.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-03\Day-11\practical-task> uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['E:\\VisionRoot-AI engineer Internship\\GitHub\\AI-Engineer-Training\\AI-Engineer-Training\\Week-03\\Day-11\\practical-task']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [23900] using WatchFiles
INFO:     Started server process [55964]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:50774 - "GET / HTTP/1.1" 200 OK
INFO:     127.0.0.1:50774 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO:     127.0.0.1:65144 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:65144 - "GET /openapi.json HTTP/1.1" 200 OK
```

Test the API documentation

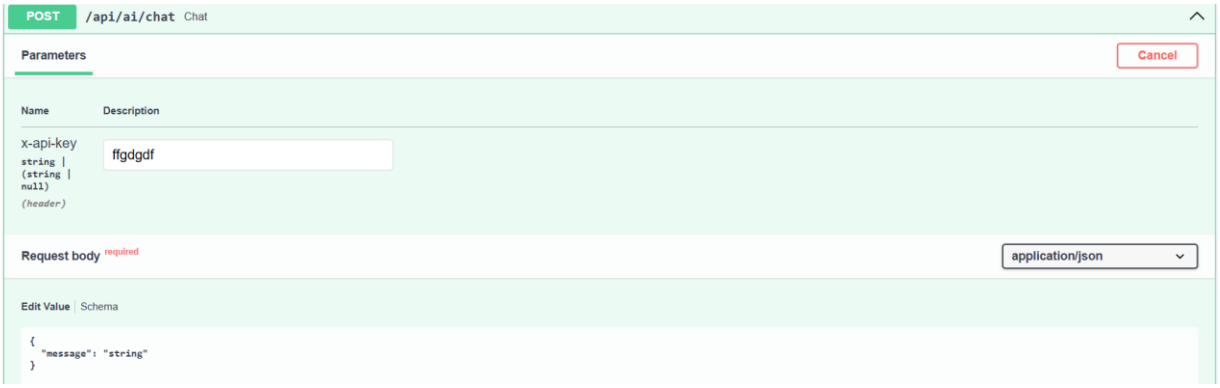


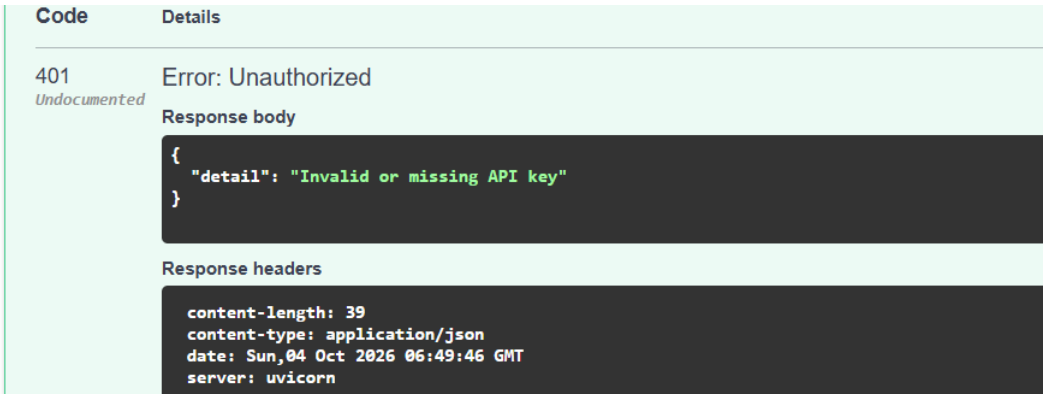
Test validation



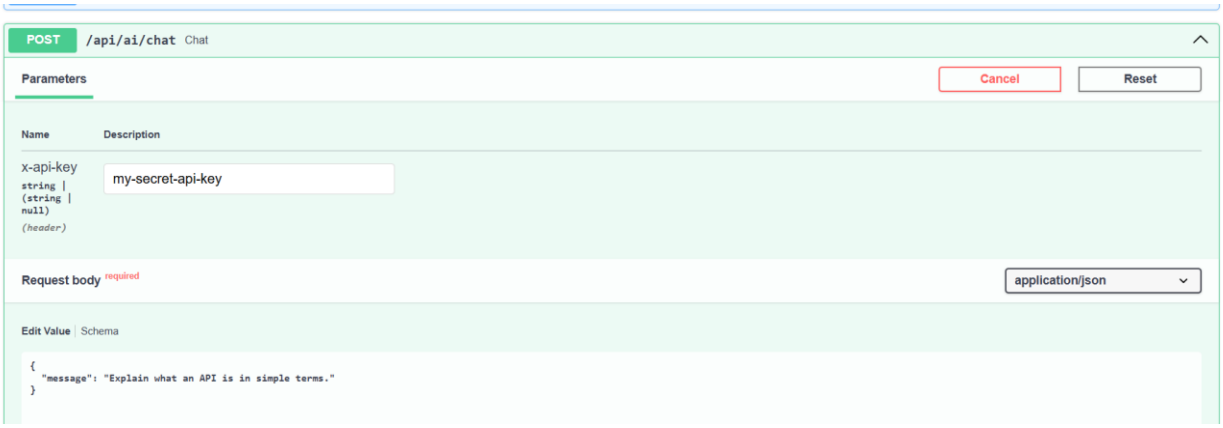


Test authentication





Test a successful request



Server response

Code	Details
200	Response body


```
{
  "success": true,
  "data": {
    "response": "Imagine you are sitting at a table in a restaurant. \n\n*   **You** are the user (or the app on your phone).\n*   **The Kitchen** is the system (like Google, Weather.com, or a bank) that has the data and does the heavy lifting.\n*   **The Waiter** is the **API**.\n\nYou look at the menu, make your choice, and tell the waiter what you want. The waiter (API) takes your order, walks back to the kitchen, and tells the chef what to cook. Once the food is ready, the waiter picks it up, brings it back to your table, and serves it to you.\n\nThe kitchen doesn't need to know who you are, and you don't need to know how to cook the food. The waiter acts as the reliable middleman that connects you to what you want.\n\n---\n\n### In Tech Terms:\n**API** stands for **Application Programming Interface**. \n\nIt is a messenger that takes requests from one software system, delivers it to another system, brings the response back, and delivers it to you.\n\n### Real-World Examples:\n\n1.  **Travel Websites (like Expedia):** When you search for flights, Expedia doesn't own any airplanes. It uses APIs to ask American Airlines, Delta, and United, *\"Show me your available seats and prices,\"* and then displays them all on one screen.\n2.  **Logging in with Google:** When a website lets you click **\"Log in with Google,\"** that website isn't seeing your Google password. It uses an API to ask Google, *\"Hey, is this person who they say they are?\"* Google replies, *\"Yep, verified,\"* and lets you in.\n3.  **Weather Apps:** Your phone's weather app doesn't have its own weather satellites. It uses an API to ask a giant meteorological database, *\"What's the temperature in Chicago right now?\"* and displays the answer on your screen.\n\n**In short:** An API is a bridge that lets different apps talk to each other easily and securely without sharing all their underlying code."
  },
  "error": null
}
```


Download

Response headers

```
content-length: 1933
content-type: application/json
date: Mon,12 Oct 2026 12:54:08 GMT
server: uvicorn
```

Responses